

Deploy Automotive App to Kubernetes

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	AMEYA DOLE	EN22CS301111
2	AMISHI TALWAR	EN22CS301112
3	ANSH MUNDRA	EN22CS301142
4	ANUJ DEVNANI	EN22CS301164
5	ARPITH SHUKLA	EN22CS301203

Group Name: Group 04D2

Project Number: DO-16

Industry Mentor Name:

University Mentor Name: Prof. Akshay Saxena

Academic Year: 2025-26

Contents

S. No.	Contents	Page No.
1.	Problem Statement & Objectives	3
1.1	Problem Statement	3
1.2	Project Objectives	3
1.3	Scope of the Project	4
2.	Proposed Solution	5
2.1	Key features	5
2.2	Overall Workflow	5
2.3	Tools & Technologies Used	7
3.	Results & Output	8
3.1	Application Output	9
3.2	Docker Build and Push Output	11
3.3	Deployment Output	12
3.4	CI/CD Pipeline Execution Output	13
4.	Conclusion	14
5.	Future Scope & Enhancements	15

1. PROBLEM STATEMENT & OBJECTIVES

1.1 Problem Statement

Modern automotive web applications require scalable, reliable, and continuously deployable architectures to handle dynamic user interactions such as browsing, filtering, and API-based data retrieval. Traditional deployment approaches often suffer from downtime during updates, lack of scalability, and inefficient resource utilization.

In this project, a lightweight automotive car showcase web application built using Flask is containerized and deployed using modern DevOps practices. The challenge lies in orchestrating the deployment on a Kubernetes cluster hosted on AWS EC2 instances while ensuring zero-downtime updates, efficient workload management, and seamless CI/CD integration. This involves designing Docker-based containerization, Kubernetes deployment and service manifests, and implementing rolling update strategies to maintain uninterrupted service availability in a cloud-based environment.

Some Key Problem Points are:

- Lack of scalable deployment for Flask-based applications
- Downtime during application updates
- Manual and inefficient deployment workflows
- Need for container orchestration and automation
- Ensuring high availability during version upgrades

1.2 Project Objectives

The primary objective of this project is to design and implement a complete DevOps pipeline for a Flask-based automotive web application, ensuring efficient deployment, scalability, and reliability using containerization and orchestration technologies.

The project focuses on integrating Docker for containerization, Kubernetes for orchestration across both local (Minikube) and cloud (AWS EC2) environments, and CI/CD tools like GitHub Actions or Jenkins for automation. Additionally, it aims to implement a zero-downtime rolling update mechanism to maintain continuous availability while deploying new versions of the application.

Specific Objectives for this project are:

- Develop a lightweight automotive web application using Flask

- Containerize the application using Docker
- Deploy the application on a local Kubernetes cluster using Minikube for testing
- Deploy the application on a Kubernetes cluster hosted on AWS EC2
- Implement CI/CD pipeline for automated build and deployment
- Achieve zero-downtime deployment using rolling updates
- Expose application via Kubernetes services for user access (local and internet)
- Ensure secure handling of environment variables and configurations

1.3 Scope of the Project

This project covers the end-to-end lifecycle of deploying a Flask-based automotive web application using modern DevOps methodologies. It includes application development, containerization, orchestration, and deployment automation across both local and cloud environments.

The scope includes deployment on a local Minikube Kubernetes cluster for development and testing, as well as on a Kubernetes cluster hosted on AWS EC2 for production-like deployment, ensuring consistency across environments.

The system supports a lightweight car showcase platform featuring functionalities such as car catalog browsing, filtering, and API-based data retrieval.

Future enhancements such as advanced monitoring, authentication systems, and improved user interface can be integrated to further extend the system capabilities but are not part of the current implementation scope.

Scope Coverage

- Flask-based web application with REST API support.
- Docker-based containerization
- Kubernetes deployment on Minikube (local testing) and AWS EC2 infrastructure
- CI/CD pipeline implementation
- Rolling update strategy for zero downtime
- Basic secrets and environment variable management

2. PROPOSED SOLUTION

2.1 Key Features

- **Interactive Car Catalog** – Supports category-based browsing with search and filter functionality for efficient user navigation
- **REST API Integration** – Provides API endpoints for dynamic data retrieval and seamless frontend-backend communication
- **Containerized Deployment** – Application is fully Dockerized, ensuring portability and consistency across environments
- **Hybrid Kubernetes Orchestration** – Uses Kubernetes on both Minikube (local testing) and AWS EC2 (cloud deployment) for managing containers, scaling, and service exposure
- **Zero-Downtime CI/CD Pipeline** – Implements automated deployment with rolling updates to ensure continuous availability without service interruption

2.2 Overall Workflow

The workflow begins with the developer pushing code to GitHub, triggering the CI/CD pipeline. A Docker image is built and pushed to Docker Hub. The deployment is then carried out in two environments: locally using Minikube for testing and on AWS EC2 using Kubernetes for cloud-based access. In both cases, Kubernetes manages pods and services to run and expose the application. Users can access the application either locally or via the internet. Any updates trigger rolling deployment, ensuring zero downtime.

The overall workflow for proposed solution for Deploying Automotive App to Kubernetes is given in Figure 2.1.

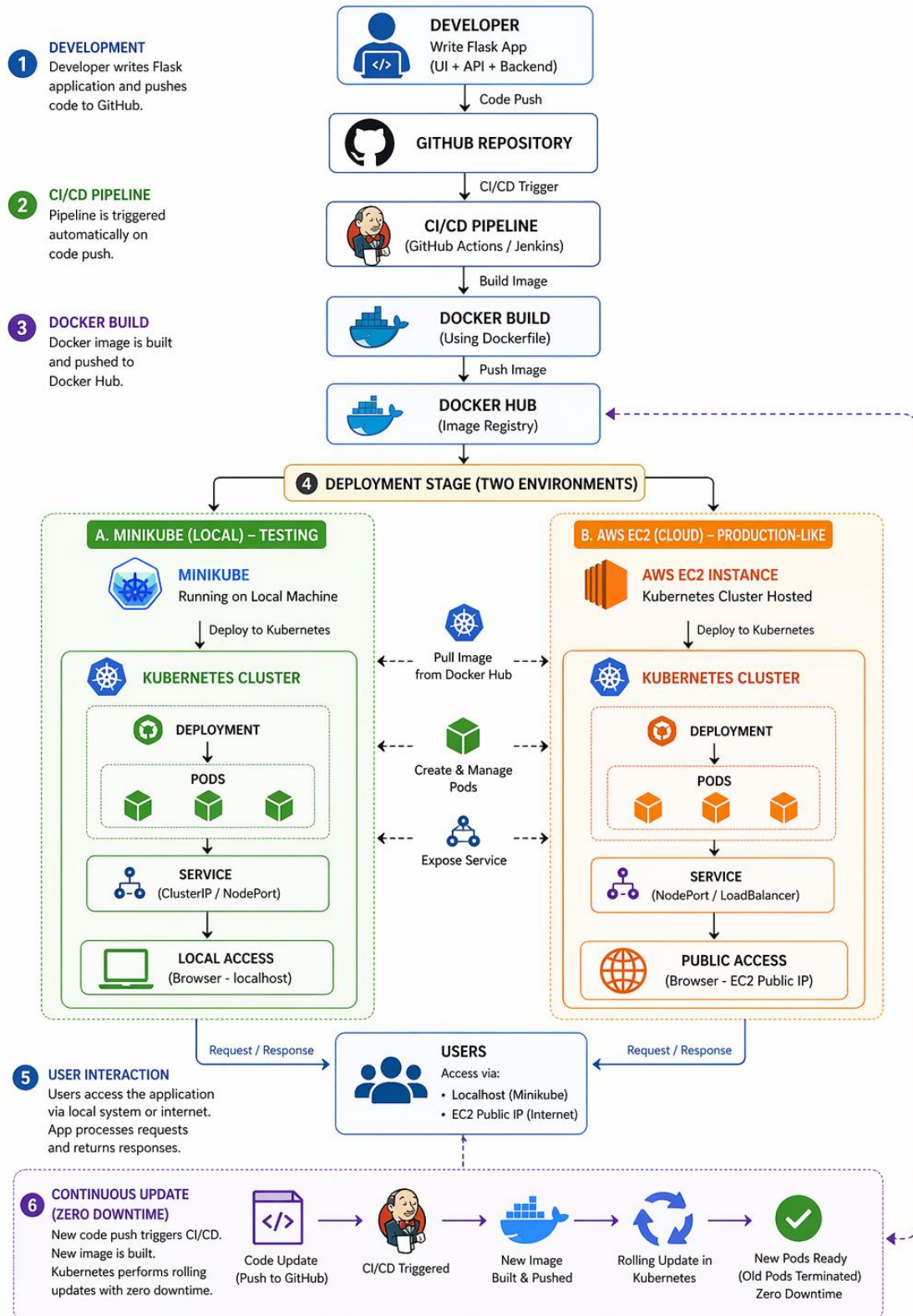


Figure 2.1: Overall Workflow for Deploy Automotive App to Kubernetes

2.3 Tools & Technologies Used

This project utilizes a combination of modern development, containerization, orchestration, and automation tools to implement a complete DevOps pipeline for the automotive web application.

1. Development & Application Layer

- **Python** – Core programming language used for backend development
- **Flask** – Lightweight web framework used to build the automotive web application and REST APIs
- **HTML** – Used for designing the frontend user interface

2. Containerization

- **Docker** – Used to containerize the Flask application ensuring portability and consistency across environments
- **Dockerfile** – Defines the environment and instructions to build the application image

3. Orchestration

- **Kubernetes (Minikube & AWS EC2)** – Manages deployment, scaling, and networking of containerized applications across both local and cloud environments
- **Minikube** – Provides a local Kubernetes cluster for development and testing purposes before cloud deployment
- **Kubernetes Deployment** – Handles pod creation, scaling, and rolling updates
- **Kubernetes Service** – Exposes the application to users via local access (Minikube) and external access (AWS EC2)
- **AWS EC2** – Provides virtual servers to host the Kubernetes cluster and enable real-world deployment

4. CI/CD & Version Control

- **Git** – Version control system to track code changes
- **GitHub** – Remote repository for code hosting
- **GitHub Actions** – Automates build, test, and deployment pipeline

5. Supporting Tools

- **pip** – Python package manager
- **VS Code** – Development environment for coding and testing
- **Docker Hub** – Stores and distributes Docker images

3. RESULTS & OUTPUTS

The output of this project demonstrates the successful implementation of a hybrid DevOps deployment pipeline for a Flask-based automotive web application. The application is containerized using Docker and deployed on Kubernetes in two environments: locally using Minikube for testing and on AWS EC2 for cloud-based, real-world accessibility.

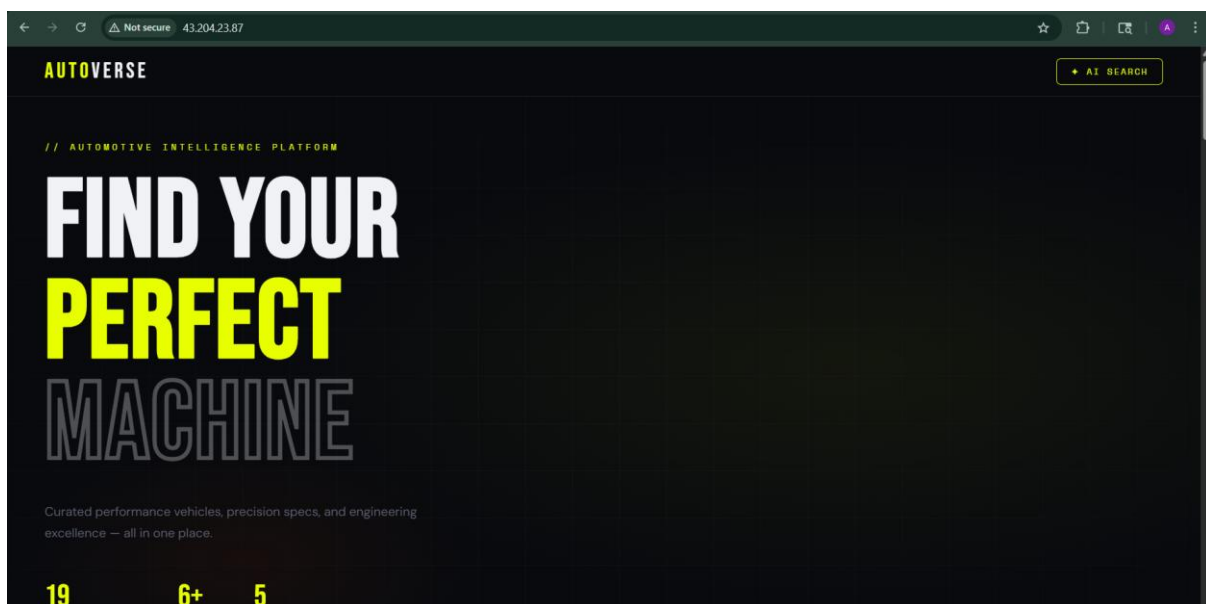
The system allows users to browse car categories, apply search and filter operations, and view detailed car information through an interactive web interface. Backend APIs provide dynamic data handling, ensuring efficient communication between frontend and server.

The CI/CD pipeline automates the build and deployment process, while Kubernetes ensures scalability and high availability. Rolling updates are implemented to achieve zero downtime during application upgrades.

3.1 Application Output

The application is successfully displayed in the browser with full functionality.

Figures 3.1 to 3.3 present the output of the developed automotive web application, showcasing the user interface, search functionality, and dynamic display of car details. The system allows users to search for vehicles based on keywords such as brand or category, and retrieves relevant results through the backend API. Each result includes key information such as specifications, performance metrics, and pricing, providing a structured and user-friendly browsing experience.



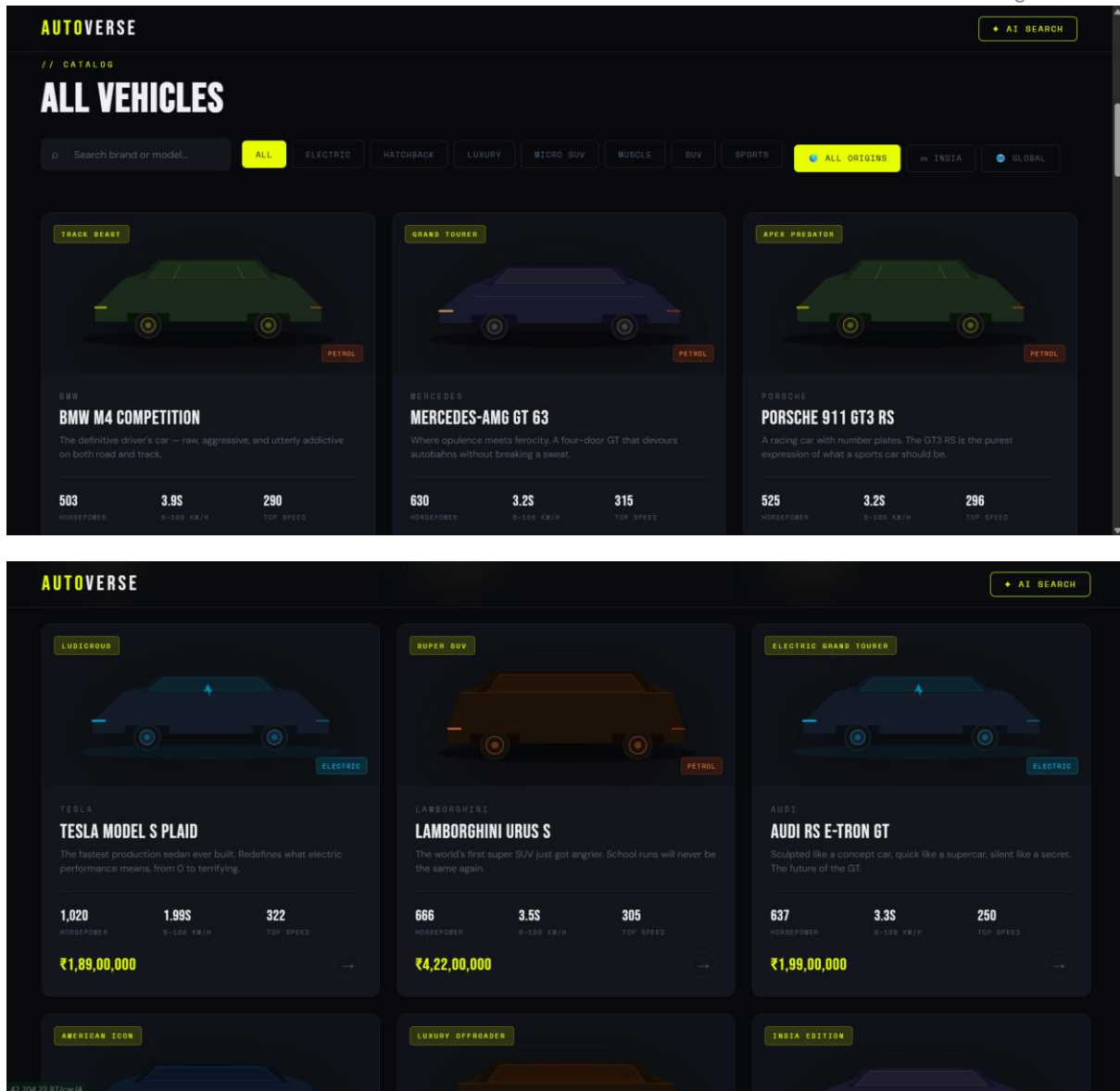


Figure 3.1-3.3: Application output

Figures 3.4 and 3.5 demonstrate the AI-powered search functionality of the application, where intelligent query processing enhances the relevance and accuracy of search results. This feature improves user interaction by delivering faster, context-aware responses, highlighting the advanced capabilities of the system beyond traditional search mechanisms.

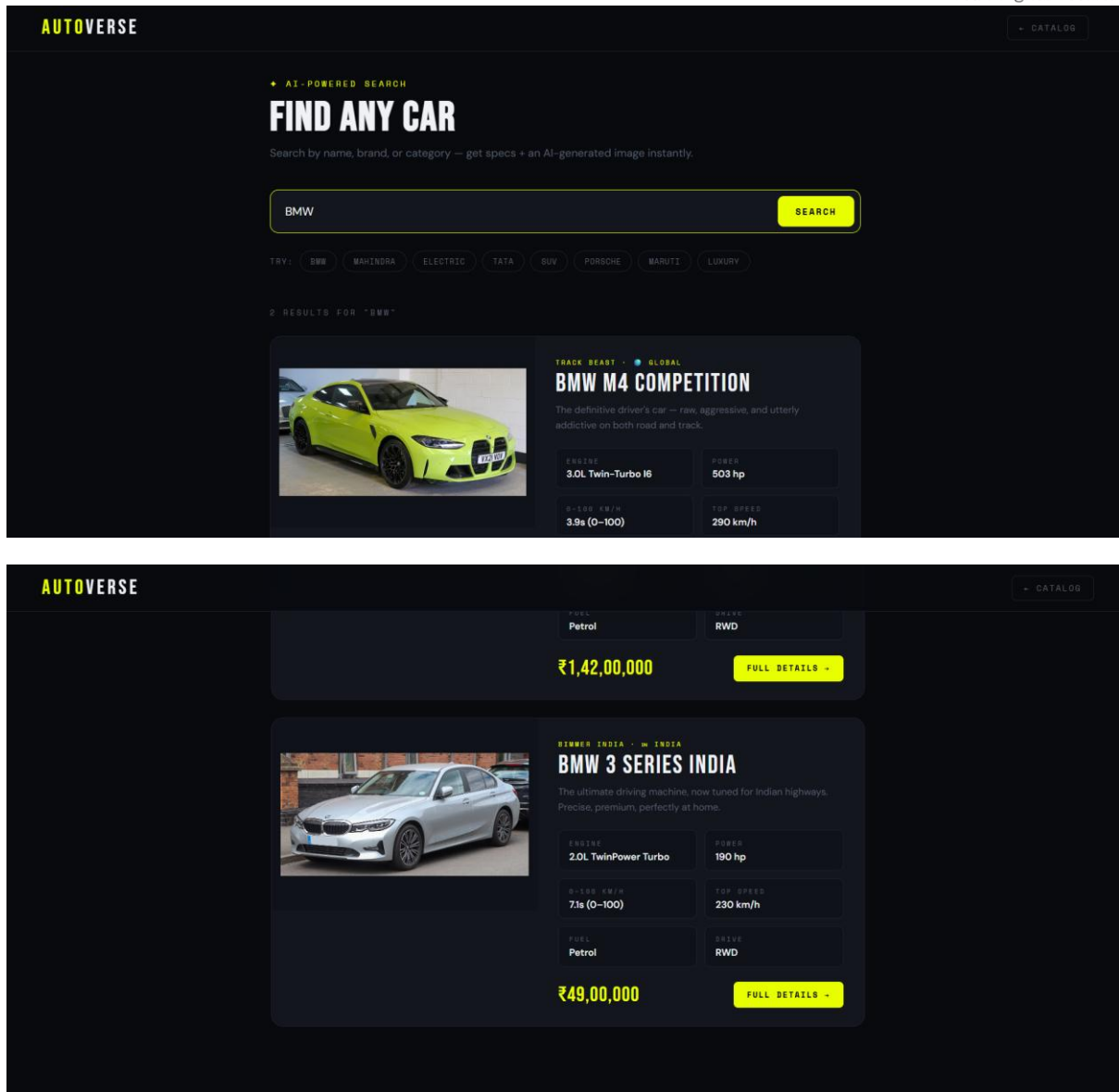


Figure 3.4-3.5: AI-powered search functionality of the application

3.2 Docker Build and Push Output

Figure 3.6 shows the successful build and push of the Docker image to Docker Hub using the CI/CD pipeline.

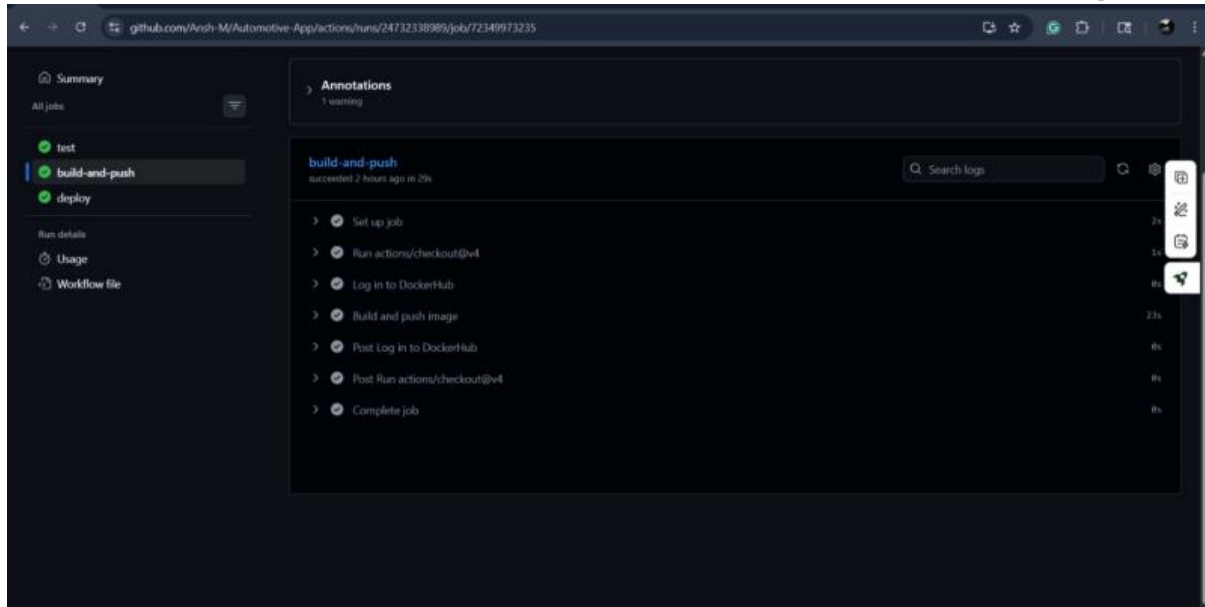


Figure 3.6: Docker image build and push process completed successfully.

3.3 Deployment Output

Figure 3.7 shows the Pods Output which shows that multiple application pods are running successfully in the Kubernetes cluster.

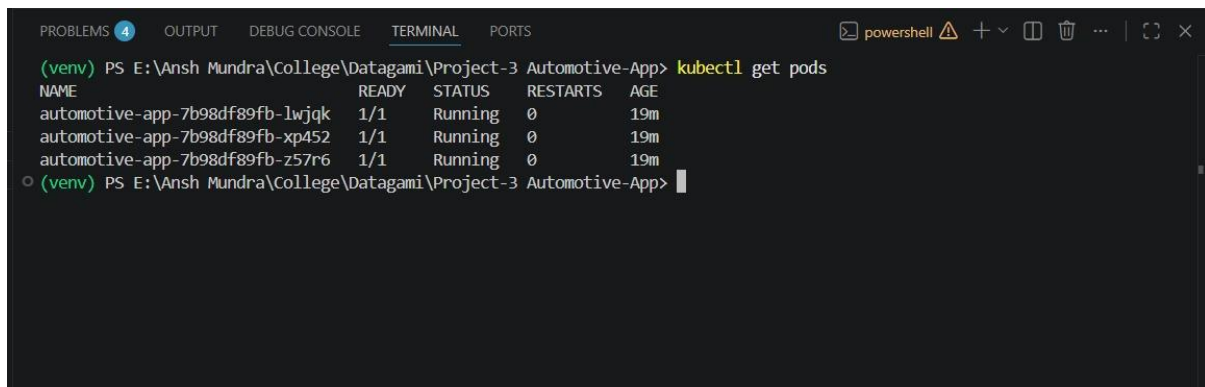


Figure 3.7: Kubernetes pods in running state

Figure 3.8 provides Service Output which shows that the application is exposed using a NodePort service for external access.

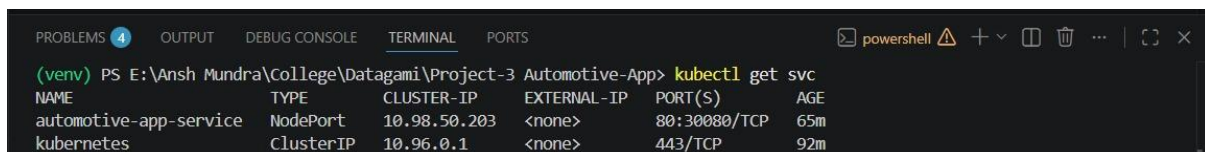


Figure 3.8: Kubernetes service exposing application via NodePort.

3.3 CI/CD Pipeline Execution Output

Figure 3.9 shows the successful execution of the CI/CD pipeline stages including testing, building, pushing and deployment using GitHub Actions.

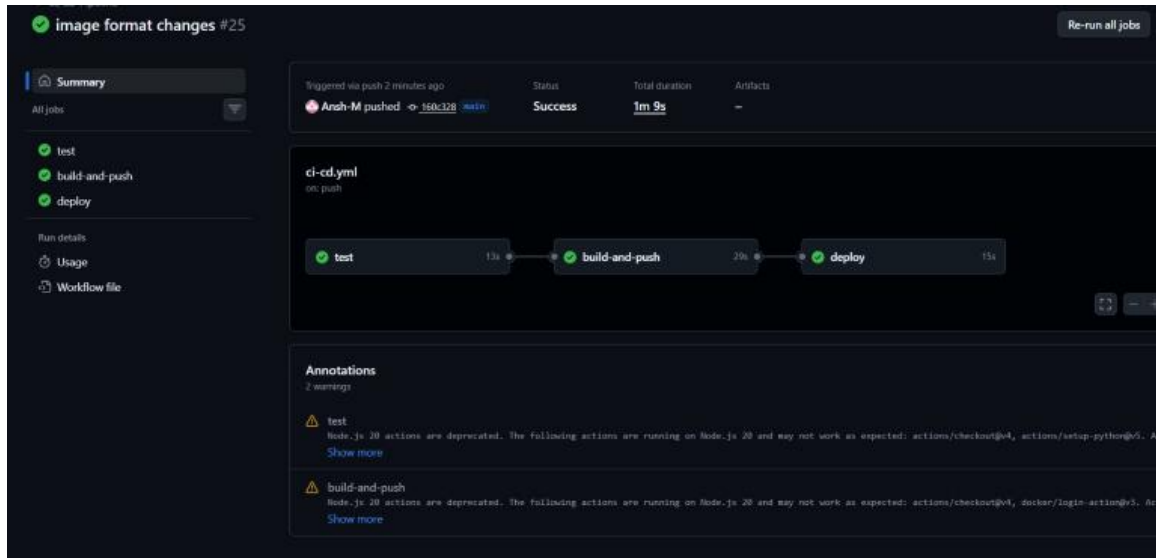


Figure 2.9: Successful execution of CI/CD pipeline stages.

4. CONCLUSION

This project successfully demonstrates the implementation of a modern DevOps-driven deployment pipeline for a lightweight automotive car showcase web application built using Flask. By integrating containerization with Docker and orchestration using Kubernetes across both local (Minikube) and cloud (AWS EC2) environments, the application achieves improved scalability, portability, and efficient resource utilization.

The use of Kubernetes deployment and service manifests enables effective management of containerized workloads in both development and production-like environments. The implementation of rolling updates ensures zero-downtime deployment, maintaining uninterrupted service availability for end-users. Additionally, the integration of CI/CD pipelines using tools like GitHub Actions automates the build and deployment process, reducing manual intervention and minimizing errors.

The hybrid deployment approach further validates how applications can be consistently tested in a local Kubernetes environment before being deployed to cloud infrastructure, ensuring reliability and smoother transitions between development and production stages.

Overall, the project demonstrates how traditional web applications can be transformed into cloud-native solutions using DevOps practices, enhancing reliability, deployment efficiency, scalability, and system resilience.

Key Takeaways are:

- Successful containerization of a Flask application using Docker
- Efficient orchestration using Kubernetes in both Minikube (local) and AWS EC2 (cloud) environments
- Zero-downtime deployment using rolling update strategy
- Automated CI/CD pipeline for continuous integration and deployment
- Improved scalability, accessibility, and maintainability of the application

5. FUTURE SCOPE AND ENHANCEMENTS

While the current implementation provides a strong foundation, there are several opportunities to enhance the system further by incorporating advanced DevOps and cloud-native features.

Since the application is deployed using a hybrid model (Minikube for local testing and AWS EC2 for cloud deployment), it can be further extended into a fully managed and production-grade environment using advanced cloud services. Enhancements such as monitoring, logging, security, and scalability can significantly improve system performance and reliability.

Application-level improvements such as authentication, personalization, and enhanced UI/UX can further improve user experience. Additionally, adopting advanced Kubernetes tools and cloud-native practices will strengthen deployment efficiency and system robustness.

Some Future Enhancements that can be done are:

- Integrate monitoring tools like Prometheus and Grafana
- Implement centralized logging using ELK Stack (Elasticsearch, Logstash, Kibana)
- Add user authentication and authorization system
- Use Helm charts for better Kubernetes deployment management
- Implement Kubernetes autoscaling (HPA) for dynamic scaling
- Strengthen security using RBAC and secret management tools
- Migrate to managed Kubernetes services (e.g., AWS EKS) for improved reliability
- Extend CI/CD pipeline to support multi-environment deployments (dev, staging, production)